COSC 111 — Final Python Project

Student Project Guidelines | 5 students per group | 8–10 minute presentation | 20 points

1. Project Objective

Identify a real-life problem and develop a working Python application that helps solve it. Demonstrate problem-solving, program design, Python fundamentals, and the ability to explain your own code.

2. Required Python Features

- Accept user input and display meaningful output.
- Use conditional statements (if / elif / else) and at least one loop (for / while).
- Create at least three meaningful user-defined functions.
- Use a list or dictionary to manage multiple items or records.
- Provide a menu with multiple operations; keep the application running until the user chooses Exit.
- Validate basic user input (e.g., invalid menu selections, negative quantities).
- Use readable variable names, logical structure, and useful comments.

Implement at least FOUR meaningful, connected, working application features. A feature is an operation the application performs—not an individual Python statement.

For example, a library application may add, search, borrow, and return books.

Demonstrate at least one complete workflow from start to finish.

A console-based application is sufficient. GUI, file storage, databases, and third-party libraries are optional, not required. Lists/dictionaries must be used meaningfully, not merely included to satisfy the requirement.

3. Suggested Project Topics

No.	Topic	Possible features
1	Student Grade Management	Add scores; calculate averages; assign grades; search students.
2	Personal Expense Tracker	Record expenses; categorize; summarize spending; search entries.
3	Library Book Manager	Add/search books; borrow/return; show availability.
4	Restaurant Ordering System	Show menu; take orders; calculate totals; print receipt.
5	Inventory Management	Add products; update quantities; search; flag low stock.
6	Parking Fee System	Record vehicles; calculate duration and fees; view records.
7	Attendance Tracker	Record attendance; calculate percentages; view student records.
8	Quiz Application	Ask questions; check answers; calculate scores; show results.
9	Electricity Bill Calculator	Enter usage; apply rates; calculate bills; compare entries.
10	Appointment Booking	View slots; book/cancel; prevent duplicate bookings.
11	Personal Budget Planner	Set budget; record income/expenses; show remaining balance.
12	Hotel Reservation System	View rooms; book/cancel; calculate accommodation costs.
13	Bank Account Simulator	Simulate deposits/withdrawals; validate balance; show history.
14	Movie Ticket Booking	View showtimes; select seats; prevent double bookings; total cost.
15	Small Business Sales Tracker	Record transactions; apply discounts; summarize revenue.

You may propose a different real-life application with instructor approval. The features above are suggestions, not a checklist. A simple application that works correctly is better than an overly complex one with broken features.

4. Group Participation

Each group must have FIVE students. All members should contribute to planning, development, testing, and presentation preparation. Every member must speak during the presentation and should understand the overall application, not only their own assigned task.

5. Group Presentation — 8–10 Minutes

Part	Suggested time
Problem, intended users, and solution	1 minute
Main features and workflow	2 minutes
Live demonstration with realistic inputs	3 minutes
Python concepts and code explanation	2 minutes
Instructor questions	1–2 minutes

All five members must speak. Have the program open and ready before your turn. The instructor may ask for additional inputs or ask a student to explain a portion of the code. Presentation arrangements and schedule will be announced on Canvas.

6. Submission — One ZIP File per Group

Submit one ZIP file through Canvas; only one member needs to upload it. Filename: GroupXX_ProjectTitle.zip (example: Group03_LibraryManager.zip). Include:

- **Python code**: main.py and any other .py files needed to run the application.
- **Presentation slides**: one PPTX or PDF with title, member names/IDs, problem and solution, main features, Python concepts, and workflow/screenshots.
- **Video demonstration**: a 3–5 minute screen recording showing the running application and at least one complete workflow. Narration is recommended.
- **README.txt**: how to run the program, any required packages, and each member's main contribution.

The recorded demo is evidence that the program works; the live demonstration lets the instructor verify functionality and understanding. Check the ZIP contents and test the program before submitting. Do not include unnecessary large files or personal data.

7. Assessment Rubric — 20 Points

Criterion	Points	What is assessed
Problem and proposed solution	2	Clear real-life need; appropriate solution.
Program functionality	5	At least four meaningful working features; complete workflow; input handling.
Python concept coverage	4	Correct, meaningful use of conditions, loops, functions, lists/dictionaries.
Code quality	2	Readable naming, organization, comments, limited repetition.
Group presentation	2	Clear slides, explanation, organization, timing.
Live program demonstration	2	Successful realistic demonstration and ability to test features.
Individual presentation and understanding	3	1 pt: explains own contribution; 1 pt: understands relevant code; 1 pt: answers questions.
TOTAL	20	17 group points + 3 individual points

8. Academic Integrity and Important Notes

You may consult documentation, tutorials, and AI tools to learn, debug, or improve your work. However, the group must understand and be able to explain all submitted code. Copying another group's project or submitting code that members cannot explain may result in reduced marks or review under course academic-integrity policy.

Topic approval, submission deadline, and presentation schedule: To be announced on Canvas.